github.com/mg-i-roc-maz/corsi · microservizi/lezione-6

LEZIONE 6 DI 8 · 3 ORE

Caching nei microservizi

Le copie si moltiplicano: nel browser, nel gateway, in ogni servizio. Chi le butta, quando un prezzo cambia?

Le tre ore di oggi

Teoria · 1h | **Laboratorio · 1h50**

Teoria

Livelli di cache, Cache-Control ed ETag, la cache del gateway, invalidazione tra servizi, misurare con k6, lo stampede.

Laboratorio

ETag e 304 in catalog, cache in nginx, k6, stampede provocato e mitigato, orders avvisato quando un prezzo cambia.

Ultimi 10'

Una domanda per chiudere: «cosa si rompe se…?»

DA DOVE RIPARTIAMO

Le tre domande con cui abbiamo chiuso la lezione 5.

TRE COPIE DI CATALOG

Cache in memoria del processo: una PUT ne aggiorna una sola.

ORDERS CON LA SUA CACHE

catalog invalida la sua copia. E quella di orders?

LO SPOT IN TV

La chiave più letta scade: migliaia di MISS insieme.

Ogni livello, una copia



Livello	Chi decide per quanto	Si può cancellare una copia?
Browser	catalog, con Cache-Control	No: è sul computer del cliente
Gateway	catalog, con Cache-Control	Con nginx open source no: solo il TTL
orders	orders, con il suo TTL	Sì, se qualcuno lo avvisa
Redis	catalog	Sì: DEL

Locale e distribuita: due livelli insieme

L1 · nel processo

- la più veloce: nessuna rete
- una per copia del servizio
- TTL di pochi secondi
- oggi: la cache di orders

L2 · condivisa

- ~0,3 ms in più per la rete
- una sola per tutte le copie
- TTL più lungo
- oggi: Redis di catalog

Si legge L1, poi L2, poi il database. Il problema è sempre lo stesso: quando il dato cambia, **chi avvisa le L1**?

CACHE HTTP

È la risposta a dire come tenerla in cache.

Il gateway e il browser non sanno niente dei prodotti. Lo sa catalog, e lo scrive negli **header** di ogni risposta.

Cache-Control

```
Cache-Control: public, max-age=10
```

Direttiva	Significato
`max-age=10`	la copia è buona per 10 secondi
`public`	la può tenere anche una cache condivisa (gateway, CDN)
`private`	solo il browser dell'utente: dati personali
`no-cache`	si può tenere, ma va **ricontrollata** ogni volta
`no-store`	non tenerla da nessuna parte
`stale-while-revalidate=30`	scaduta, si usa ancora per 30 s mentre la si rinnova

ETag e 304: «è ancora questa?»

browser	`GET /products/p-1`	`200 · ETag: "Wjna…" · 99 byte`
browser	`…10 secondi dopo, la copia è scaduta`	
browser	`GET · If-None-Match: "Wjna…"`	`304 Not Modified · 0 byte`

L'ETag

L'**impronta** del contenuto: un hash del JSON. Cambia il prezzo, cambia l'ETag.

Il 304

«Hai già la versione giusta»: niente corpo. Si risparmia banda, non la richiesta.

public o private?

public

Uguale per tutti: un prodotto, una pagina del catalogo.

Il gateway tiene **una** copia e la dà a tutti.

private

Di un solo utente: un ordine, un carrello, un profilo.

Solo il browser di quell'utente può tenerla.

Un ordine con **public** dietro una cache condivisa: il primo cliente che lo legge lo fa vedere a tutti quelli che chiedono lo stesso indirizzo. È una fuga di dati, non un problema di prestazioni.

La cache nel gateway

```
proxy_cache_path /var/cache/nginx/shop
  keys_zone=products:10m max_size=50m
  inactive=10m use_temp_path=off;

location /api/v1/products {
  proxy_pass http://catalog:3001/products;
  proxy_cache products;
  add_header X-Gateway-Cache
             $upstream_cache_status always;
}
```

La chiave è l'indirizzo completo: **/api/v1/products/p-1**, **/api/v1/products?q=mouse**.

Per quanto tenere la copia: lo dice il **Cache-Control** di catalog.

Si tengono solo GET e HEAD.

Da dove arriva la risposta?

X-Gateway-Cache	Significato
`MISS`	non c'era: chiesta a catalog, ora salvata
`HIT`	servita dalla copia, catalog non è stato chiamato
`EXPIRED`	c'era ma scaduta: chiesta di nuovo a catalog
`STALE`	scaduta, servita lo stesso (catalog giù o rinnovo in corso)
`UPDATING`	scaduta, servita mentre un'altra richiesta la rinnova
`REVALIDATED`	scaduta, catalog ha detto 304: la copia era ancora buona
`BYPASS`	la cache è stata saltata per regola

Una copia porta gli header della prima volta

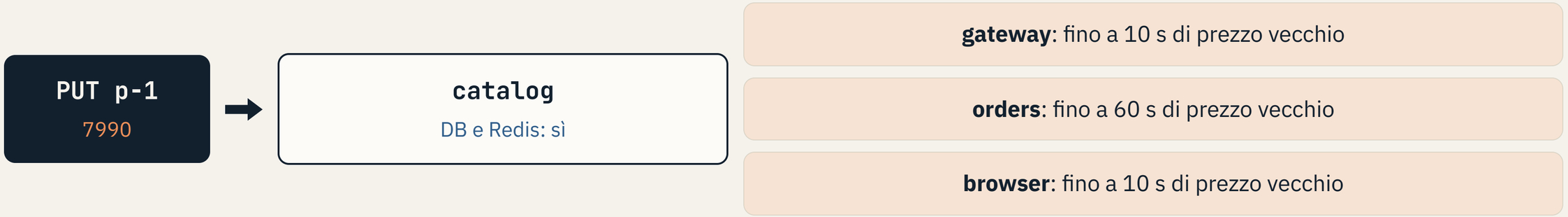
```
# senza correzione: tutte le risposte in cache
# hanno lo stesso X-Request-Id, quello del MISS

proxy_hide_header X-Request-Id;
add_header X-Request-Id $request_id always;
```

La copia salvata contiene **tutti** gli header della risposta di catalog, compreso il correlation ID della lezione 4.

Mille clienti, lo stesso ID: il grep nei log non funziona più.

Il prezzo cambia: chi lo sa?



catalog cancella le copie che **conosce**. Le altre vivono fino al loro TTL, a meno che qualcuno non le **avvisi**.

Avvisare chi ha una copia




```
{ "type": "ProductChanged", "productId": "p-1",
  "change": "updated", "at": "2026-10-20T10:15:02Z" }
```

catalog non sa chi ascolta, né quanti sono: **pubblica** e basta. Chi è iscritto al canale riceve il messaggio.

Chi ascolta decide cosa gli interessa

change	Quando	orders butta la copia?
updated	PUT: nome, prezzo, descrizione	**sì**: il prezzo può essere cambiato
deleted	DELETE	**sì**: il prodotto non esiste più
stock	una prenotazione	**no**: orders non usa le scorte della copia

⚠ Senza il campo **change**, ogni ordine (che prenota scorte) butterebbe la copia appena salvata. Misurato: hit ratio **zero**.

Redis Pub/Sub: spara e dimentica

Cosa garantisce

- arriva subito a chi è collegato
- catalog non conosce gli iscritti
- già nel Compose: niente di nuovo

Cosa NON garantisce

- orders spento o scollegato: **perso**
- nessuna conferma di lettura
- nessun nuovo tentativo

Per questo la copia di orders ha **anche** un TTL. L'avviso accorcia l'attesa; il TTL la limita quando l'avviso si perde.

Misurare sotto carico: k6

```
import http from "k6/http";

export const options = {
  vus: 20, // utenti virtuali
  duration: "20s",
};

export default function () {
  http.get(`${BASE}/products/p-1`);
}
```

Nel riepilogo	Cosa dice
http_reqs	richieste al secondo
http_req_duration	quanto dura una richiesta
p(95)	il 95% sta sotto questo tempo
checks	quante risposte erano giuste

Ogni VU ripete la funzione senza pause, finché il test non finisce.

Perché il 95° percentile e non la media

Richieste	Tempo
95 su 100	2 ms
5 su 100	300 ms (MISS)
media	**16,9 ms**
p(95)	**2 ms**
p(99)	**300 ms**

La media mescola tutto e non descrive **nessuna** richiesta vera.

I percentili dicono cosa vive il cliente: **p(95)** il caso normale peggiore, **p(99)** la coda.

Un cliente che apre 20 pagine incontra quasi sempre il p(99).

I numeri del laboratorio

k6, 20 utenti, 20 s	Richieste al secondo	p(95)
Attraverso il gateway, con la cache	~13.600	4,3 ms
Direttamente a catalog (Redis)	~7.300	6,4 ms

Stampede: 50 utenti, 30 s, TTL 5 s, DB 300 ms	Letture dal database
Senza single flight	300
Con single flight	6

Misurati sulla soluzione: i vostri numeri saranno diversi, i **rapporti** no.

Lo stampede, e il single flight

```
const running = inFlight.get(key);
if (running) {
  stats.coalesced++;
  return await running; // aspetta quella
}
const reading = load(store, id, key);
inFlight.set(key, reading);
try {
  return await reading;
} finally {
  inFlight.delete(key); // anche se fallisce
}
```

Cinquanta MISS sulla stessa chiave, nello stesso istante: la prima va al database, le altre **aspettano la sua risposta**.

Una **Promise** condivisa: in Node basta una Map.

Contro lo stampede, a ogni livello

Dove	Rimedio	Come
catalog	single flight	una lettura per chiave, le altre aspettano
gateway	proxy_cache_lock on	una richiesta va a catalog, le altre aspettano
gateway	use_stale updating	chi arriva durante il rinnovo riceve la copia scaduta
risposta	stale-while-revalidate	lo stesso, deciso da catalog per tutte le cache (extra 1)
TTL	jitter	chiavi salvate insieme non scadono insieme (extra lezione 5)

Due famiglie: **far aspettare** (lock, single flight) o **servire il vecchio** mentre si rinnova (stale).

Verifichiamo: cache HTTP e gateway

1. Che differenza c'è tra no-cache e no-store?
2. Un 304 fa risparmiare la richiesta o la banda?
3. Perché nel gateway non possiamo cancellare una copia dopo una PUT?
4. Perché un ordine non deve essere public?

Verifichiamo: invalidazione e stampede

1. orders è spento quando catalog pubblica ProductChanged: cosa succede?
2. Perché orders ignora gli avvisi con change: stock?
3. Cosa fa il single flight, e perché serve il finally?
4. Perché si guarda il p(95) e non la media?

Glossario

Termine	Significato
Cache-Control	Header con le regole: per quanto, e chi può tenere la copia.
ETag, If-None-Match	L'impronta del contenuto, e la domanda «è ancora questa?».
304 Not Modified	«La tua copia va bene»: risposta senza corpo.
Cache condivisa	Una cache usata da più utenti: gateway, CDN, proxy.
L1, L2	Cache nel processo e cache condivisa, una davanti all'altra.
Publish/subscribe	Chi pubblica non sa chi ascolta; gli iscritti ricevono.
Single flight	Una sola lettura per chiave, le altre aspettano quella.
Percentile p95	Il tempo entro cui risponde il 95% delle richieste.

Laboratorio

20'

1 · ETag e 304

TODO 1 in catalog.

20'

2 · Gateway

TODO 2: la cache di nginx.

15'

3 · k6

Misurare sotto carico.

25'

4 · Stampede

TODO 3 e 4: provocato e mitigato.

20'

5 · Tra servizi

TODO 5 e 6: avvisare orders.

```
cd microservizi/lezione-6/start → docker compose up --build -d
```

Il repo della lezione

`github.com/mg-i-roc-maz/corsi`

```
cd corsi
git pull
cd microservizi/lezione-5/start
docker compose down
cd ../../lezione-6/start
docker pull grafana/k6:1.0.0
docker compose up --build -d
```

microservizi/lezione-6/

Cartella	Contenuto
`slides.pdf`	Queste slide, in PDF
`start/`	**Si lavora qui**
`solution/`	La soluzione, per riallinearsi
`extra/`	Esercizi extra, test e soluzioni

Nuova dipendenza in orders: **redis**. **npm install** in catalog e in orders.

Cosa c'è in start

```
gateway/nginx.conf                TODO 2, 4
services/catalog/src/
  http-cache.ts                   TODO 1
  product-cache.ts                TODO 3, 5
  notifier.ts                     PUBLISH su Redis
services/orders/src/
  local-cache.ts                  copia locale dei prodotti
  invalidation.ts, server.ts      TODO 6
k6/carico.js, scripts/            carico e stampede
```

È la soluzione della lezione 5, più le copie fuori da catalog.

Da leggere prima: **local-cache.ts**, il «problema» della Parte 5.

1.1 · TODO 1: Cache-Control ed ETag

In **sendCacheable**, dentro src/http-cache.ts:

1. calcola l'ETag con **etagOf(body)**
2. header **Cache-Control**: public, max-age=…
3. header **ETag**
4. If-None-Match corrisponde (**matches**)? **304** senza corpo
5. altrimenti il corpo

NPM TEST IN CATALOG, ALL'INIZIO

```
# pass 28 · # fail 7
```

DOPO IL TODO 1

```
# fail 3
```

1.2 · Guarda gli header

```
docker compose up --build -d
curl -i localhost:8080/api/v1/products/p-1
# HTTP/1.1 200 OK
# cache-control: public, max-age=10
# etag: "WjnaC8NThjPQHTtZ87UAELCSTJ8"

curl -i -H 'If-None-Match: "WjnaC8N..."' \
  localhost:8080/api/v1/products/p-1
# HTTP/1.1 304 Not Modified
```

Copia l'ETag **con le virgolette**.

Poi cambia il prezzo con la PUT di shop.http e riprova con la stessa ETag: cosa ricevi?

2.1 · TODO 2: la cache del gateway

```
location /api/v1/products {
  proxy_pass http://catalog:3001/products;
  ...
  proxy_cache products;
  add_header X-Gateway-Cache
   $upstream_cache_status always;
}
```

```
docker compose restart gateway
./scripts/misura.sh
```

La zona **products** è già definita da **proxy_cache_path**, in alto.

use_stale, background_update e revalidate sono già scritti: senza proxy_cache non fanno niente.

2.2 · Due cache, una dietro l'altra

```
./scripts/misura.sh
# # gateway catalog ms
# 1 MISS MISS 109
# 2 HIT MISS 0
# 3 HIT MISS 1
# ... 11 secondi dopo:
# 1 STALE MISS 4
# 2 HIT MISS 0
```

Con un HIT del gateway catalog **non viene chiamato**: la colonna catalog mostra l'header salvato nella copia.

Poi: PUT del prezzo di p-1 e GET dal gateway. Per quanto vedi il prezzo vecchio?

3 · k6: gateway o catalog?

```
docker compose run --rm k6 run /scripts/carico.js

docker compose run --rm k6 run \
  -e BASE=http://catalog:3001 /scripts/carico.js
```

Il primo passa dal gateway (con la cache), il secondo va dritto a catalog (Redis).

Variabile	Default
BASE	http://gateway/api/v1
ID	p-1
VUS	20
DURATION	20s

Confronta **http_reqs** e il **p(95)** di http_req_duration.

4.1 · Provoca lo stampede

```
CACHE_TTL_SECONDS=5 DB_DELAY_MS=300 \
  docker compose up -d catalog
./scripts/stampede.sh
# ...
# letture dal database durante il test: 300
```

50 utenti per 30 secondi sulla stessa chiave, **direttamente a catalog**: niente gateway davanti.

La chiave scade ogni 5 secondi. Quante letture ti aspetti? Quante ne vedi?

4.2 · TODO 3 e 4: una lettura per volta

TODO 3 · catalog

In **getProduct**, src/product-cache.ts:

1. lettura in corso in **inFlight**? aspettala
2. altrimenti **load(...)** e registrala
3. **finally**: toglila dalla Map

```
CACHE_TTL_SECONDS=5 DB_DELAY_MS=300 \
  docker compose up --build -d catalog
./scripts/stampede.sh
```

TODO 4 · gateway

```
proxy_cache_lock on;
proxy_cache_lock_timeout 5s;
```

Lo stesso rimedio, davanti a catalog: con dieci MISS insieme nel gateway, una sola richiesta va avanti.

5.1 · Il prezzo vecchio, nell'ordine

```
# da http/shop.http
POST /api/v1/orders # p-1: 8990
PUT /api/v1/products/p-1 # priceCents 7990
POST /api/v1/orders # p-1: ???
GET /stats/orders-cache
```

orders tiene una copia di nome e prezzo per **60 secondi**.

Domanda

Che prezzo ha la riga del secondo ordine? Chi ci perde?

5.2 · TODO 5 e 6: avvisare orders

TODO 5 · catalog

In **invalidate**, src/product-cache.ts:

`await events?.productChanged(id, change);`

in un try/catch: un avviso perso non annulla la modifica.

TODO 6 · orders

concernsOrders: «stock» non interessa.

In **server.ts**: se interessa, **catalog?.invalidate(id)** e una riga di log.

```
docker compose up --build -d
docker compose exec redis redis-cli SUBSCRIBE catalog.products
```

Checkpoint

Verifica	Esito atteso
`npm test in catalog e in orders`	tutto verde, alcuni saltati
`curl -i con If-None-Match`	304 Not Modified
`./scripts/misura.sh`	MISS del gateway, poi HIT in 1–2 ms
`k6 gateway / catalog`	circa il doppio delle richieste al secondo
`stampede.sh, TTL 5 s, DB 300 ms`	~6 letture invece di ~300
`ordine, PUT del prezzo, ordine`	il secondo ordine ha il prezzo nuovo

Consegna: i due riepiloghi di k6 e i due numeri di stampede.sh.

Se qualcosa non va

Sintomo	Causa e rimedio
`Sempre MISS nel gateway`	Manca Cache-Control: TODO 1, o catalog non ricostruito
`X-Gateway-Cache non compare`	TODO 2 non fatto, o manca restart gateway
`304 mai, sempre 200`	ETag copiata senza virgolette
`k6: lookup gateway`	Il negozio non è avviato: up -d prima
`stampede.sh: sempre ~300`	catalog non ricostruito: --build
`Tutto si blocca dopo un errore del DB`	Manca il finally nel single flight
`Il secondo ordine ha il prezzo vecchio`	TODO 5 o 6; orders senza REDIS_URL

ULTIMI 10 MINUTI

Cosa si rompe se...?

...orders si riavvia proprio mentre catalog pubblica tre cambi di prezzo?

...mettiamo max-age=86400 sui prodotti per far contento k6?

...invece di un avviso, catalog volesse dire a notifications «manda l'email per l'ordine 42»?

PROSSIMA LEZIONE

Lezione 7 · Architetture event-driven

RabbitMQ nel Compose: orders pubblica OrderCreated, un nuovo servizio notifications lo consuma. Code, exchange e conferme, nella Management UI.

Per la volta prossima: npm test verde e il secondo ordine con il prezzo nuovo.

PER CHI FINISCE PRIMA, O PER CASA

Esercizi extra

#	Esercizio	Livello	Dove
1	Una copia scaduta, ma subito	base	catalog
2	Dopo una riconnessione	base	orders
3	La cache privata degli ordini	medio	orders
4	Due livelli in catalog	avanzato	catalog
5	Quante richieste arrivano al database?	carta e penna	—

Testi in lezione-6/extra/README.md. Per 1–4 ci sono test pronti; le soluzioni sono in lezione-6/extra/soluzione.

Una copia scaduta, ma subito

Dopo max-age, per altri N secondi si può usare la copia vecchia mentre la si rinnova.

```
Cache-Control: public, max-age=10,
  stale-while-revalidate=30
```

- parametro di **sendCacheable**
- variabile **HTTP_SWR_SECONDS**

Domanda

Perché per un prezzo 30 secondi possono essere troppi, e per una descrizione no?

Dopo una riconnessione

Redis Pub/Sub non conserva i messaggi: gli avvisi arrivati mentre orders era scollegato sono persi.

- **clear()** in local-cache.ts
- opzione **onConnected**, all'evento **ready**
- in server.ts: a ogni connessione, clear()

Prova

Ordine, **stop redis**, PUT del prezzo, **start redis**, ordine. Che prezzo ha?

La cache privata degli ordini

Un ordine è un dato personale: va bene nel browser di chi l'ha fatto, **mai** in una cache condivisa.

```
Cache-Control: private, no-cache
ETag: "..."
```

Con If-None-Match uguale: 304.

Domanda

Che succede se per errore è **public, max-age=60** e il gateway tiene in cache anche gli ordini?

Due livelli in catalog

Una cache locale (L1) di pochi secondi davanti a Redis (L2): **X-Cache: LOCAL**.

- opzione **localTtlMs**, variabile **CACHE_LOCAL_TTL_MS**
- una PUT butta anche la copia locale
- le altre copie di catalog? iscrivile al canale

```
CACHE_LOCAL_TTL_MS=5000 docker compose up -d \
  --scale catalog=3
```

Il punto

La PUT arriva a **una** copia di catalog. Come lo sanno le altre due?

Quante richieste arrivano al database?

10.000 letture in un minuto. Il browser ne serve il 40%, il gateway il 70% di quelle che gli arrivano, Redis il 95%.

1. Quante al gateway, a catalog, al database?
2. Tempi 0 / 1 / 3 / 100 ms: la media?
3. Il gateway perde la cache: e il database?

Infine

Conviene migliorare il 70% del gateway o il 95% di Redis?

Per approfondire

MDN · *HTTP caching*	developer.mozilla.org: Cache-Control, ETag, richieste condizionali
RFC 9111 · *HTTP Caching*	lo standard, per chi vuole i dettagli
nginx · *ngx_http_proxy_module*	proxy_cache, proxy_cache_lock, use_stale
Grafana k6 · documentazione	grafana.com/docs/k6: scenari, metriche, soglie
Redis Docs · *Pub/Sub*	SUBSCRIBE, PUBLISH e cosa (non) garantiscono
M. Kleppmann · *Designing Data-Intensive Applications*	cap. 11: flussi di eventi e dati derivati